```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Updated to a working URL for the Mall Customers dataset
url = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning-with-Python/master/Datasets/

df = pd.read_csv(url)

print("First five records:")
display(df.head())

print("Dataset shape:", df.shape)
```

```
First five records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
Dataset shape: (200, 5)
```

```
print("Dataset information:")
df.info()

print("\nMissing values:")
print(df.isnull().sum())

print("\nStatistical summary:")
display(df.describe())

print("\nDuplicate records:", df.duplicated().sum())
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

Missing values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Statistical summary:
```

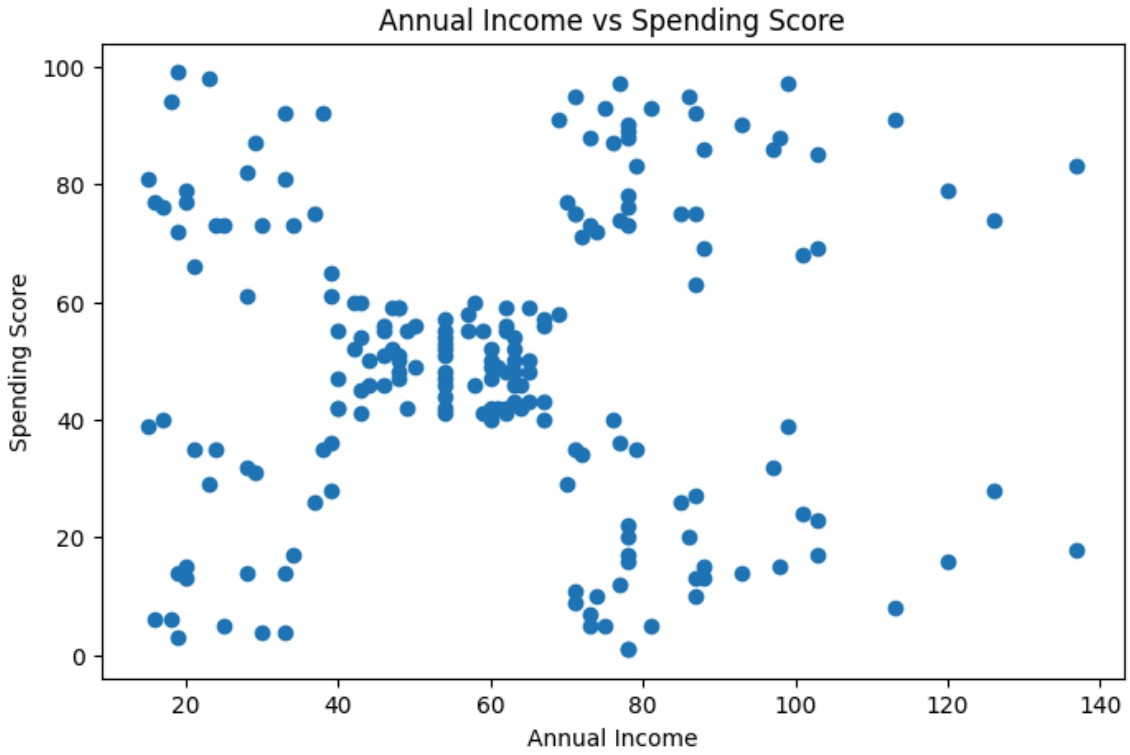
	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
Duplicate records: 0
```

```
plt.figure(figsize=(8, 5))

plt.scatter(
    df["annual_income_"],
    df["spending_score_"]
)

plt.title("Annual Income vs Spending Score")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.show()
```




```
X = df[["annual_income_", "spending_score_"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Standardized data:")
display(pd.DataFrame(
    X_scaled,
    columns=["Annual Income", "Spending Score"]
).head())
```

Standardized data:

	Annual Income	Spending Score
0	-1.738999	-0.434801
1	-1.738999	1.195704
2	-1.700830	-1.715913
3	-1.700830	1.040418
4	-1.662660	-0.395980

```
# Standardize column names for consistent use
df.columns = [
    col.strip().lower().replace(" ", "_")
    .replace("(k$)", "")
    .replace("(1-100)", "")
    .replace("(", "").replace(")", "")
    for col in df.columns
]

print(df.columns.tolist())
```

```
['customerid', 'gender', 'age', 'annual_income_', 'spending_score_']
```

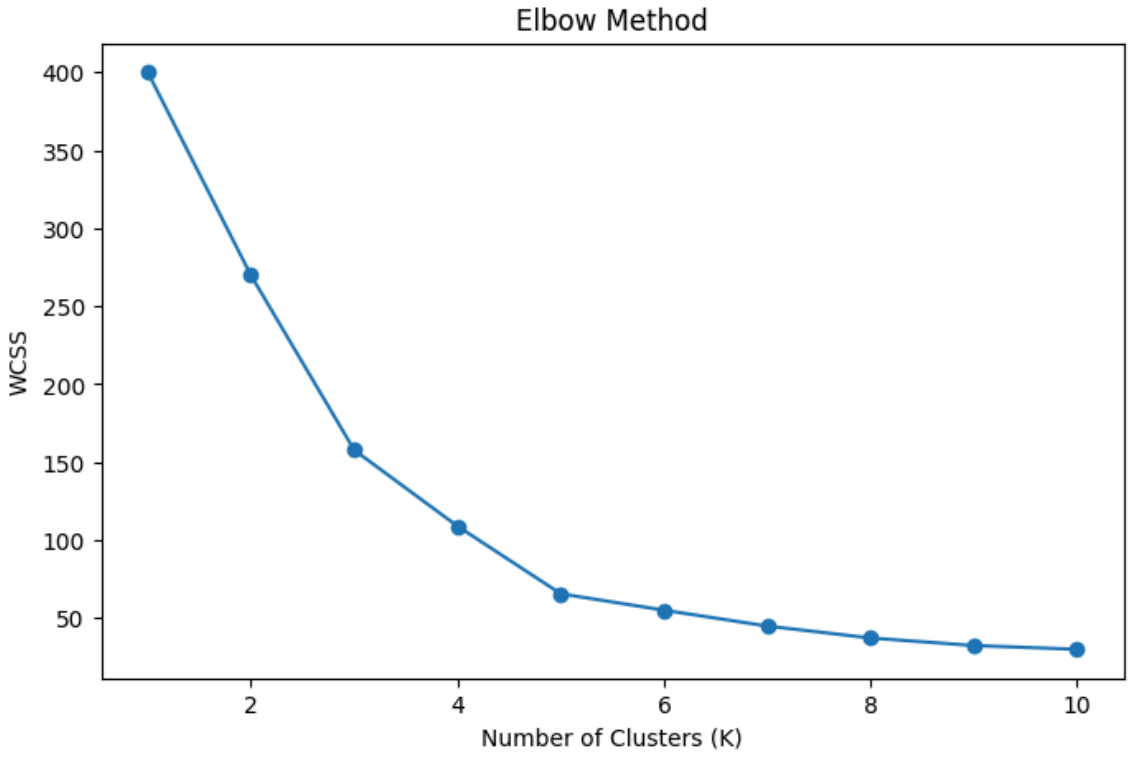
```
# Use these two features for clustering
X = df[["annual_income_", "spending_score_"]]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

wcss = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
```




```
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X_scaled)

print("Cluster assignments:")
display(df.head(10))
```

```
Cluster assignments:
```

	customerid	gender	age	annual_income_	spending_score_	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	2
8	9	Male	64	19	3	4
9	10	Female	30	19	72	2

```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="annual_income_",
    y="spending_score_",
    hue="Cluster",
    palette="viridis",
    s=80
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.legend(title="Cluster")
plt.show()
```

```
centers = scaler.inverse_transform(kmeans.cluster_centers_)

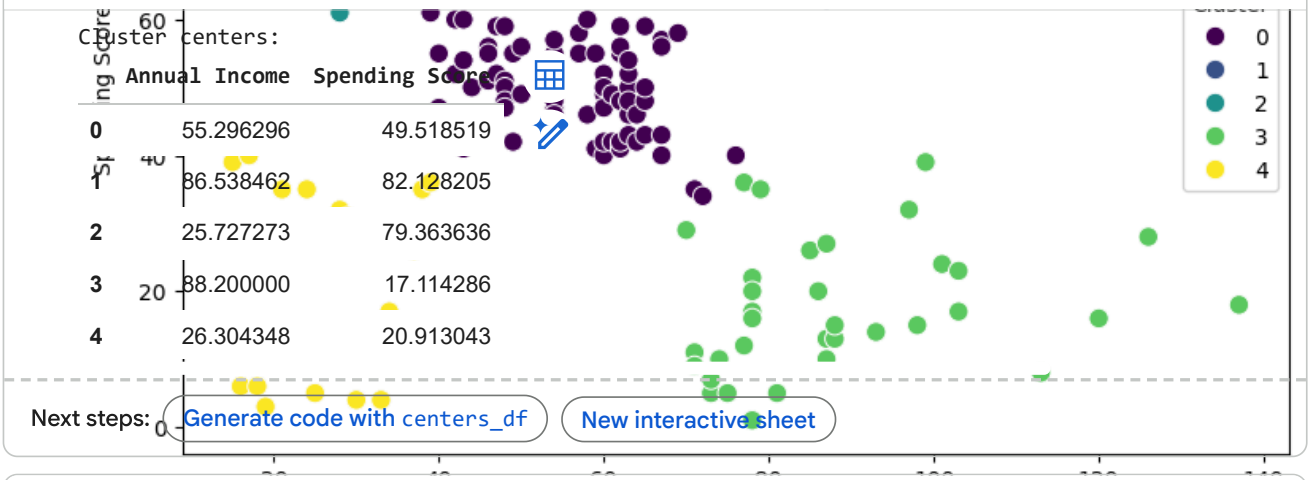
centers_df = pd.DataFrame(
    centers,
    columns=["Annual Income", "Spending Score"]
)

print("Cluster centers:")
display(centers_df)
```

```
Cluster centers:
```

	Annual Income	Spending Score
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

Next steps: Generate code with centers_df | New interactive sheet




```
print("Number of customers in each cluster:")
print(df["Cluster"].value_counts().sort_index())

print("\nAverage values by cluster:")
display(
    df.groupby("Cluster")[
        ["annual_income_", "spending_score_"]
    ].mean().round(2)
)
```

```
Number of customers in each cluster:
Cluster
0    81
1    39
2    22
```